

CSE 4304-Data Structures Lab. Winter 2024-25

Date: 25 February 2026

Target Group: All groups

Topic: Range Queries: Prefix Sum, Sparse Table, and Fenwick Tree.

Instructions:

- Task naming format: fullID_T01L01_2A.c/cpp
- If you find any issues in the problem description/test cases, comment in the Google Classroom.
- If you find any tricky test cases that I didn't include but that others might forget to handle, please comment! I'll be happy to add them.
- Use appropriate comments in your code. This will help you recall the solution more easily in the future.
- The obtained marks will vary based on the efficiency of the solution.
- Do not use <bits/stdc++.h> library.
- Modified sections will be marked with **BLUE** color.
- You are allowed to use the STL stack unless it's specifically mentioned to use manual functions.

Group	Tasks
1A/1B/2A/2B	
Assignment	All tasks in this document

Task 1: THRBL - Catapult that ball

Bob has an unusual problem. In Byteland, there are many hills and cities. The king of Byteland ordered Bob to deliver magic balls from one city to another. Unfortunately, Bob has to deliver many magic balls, so walking with them would take too much time for him. Bob came up with a great idea—catapulting them.

Byteland is divided into intervals. Each interval contains cities and hills.

Bob can catapult a magic ball accurately from city A to city B if there isn't a higher hill between them than A's hill.

Input

- Every test case contains N and M ($N \leq 50000$, $M \leq 50000$), representing the number of intervals, and number of balls.
- In the next line, there are N numbers H ($H \leq 109$) separated by one space.
- In the next M lines numbers A and B ($1 \leq A, B \leq N$), the city from which we want to catapult the ball and the city to which we want to catapult the ball.

Output

Write one number - the number of magic balls that Bob can catapult successfully.

Sample Input	Sample output	Explanation
7 3 2 3 5 4 2 1 6 3 5 2 5 4 6	2	Bob can catapult balls numbered 1 and 3. [2 5] not possible as there is a hill of height 5 which is larger
6 4 4 2 6 1 3 5 1 3 2 4 5 6 1 6	2	[2 4] and [1 6] not possible as there is a bigger hill in the middle
4 2 5 5 5 5 1 4 2 3	2	

Note: We are using 1-based indexing. So [5 4 2] is working with value 4 2 1, meaning index 3 to 5 of the original array.

Task 2: (10 points) Bit Flipper

What is the minimum number of bits you need to flip (change 0 to 1 or 1 to 0) to make the subarray contain an equal number of 0s and 1s?

Input:

You are given a binary array A of length n (containing only 0s and 1s). You need to answer Q queries, where each query is a subarray [L,R] (1 indexed) and you must determine:

Output:

For each query, the number of flips required if possible. (-1 if impossible)

Sample Input	Sample output	Explanation
6 3 0 1 0 1 1 0 1 4 2 5 3 6	0 1 0	
5 2 0 0 0 0 0 1 5 2 4	-1 -1	
4 2 0 1 0 1 1 4 2 3	0 0	
1 1 0 1 1	-1	
7 2 1 1 1 1 1 0 0 1 4 3 7	2 -1	
20 10 1 0 1 1 0 0 1 1 0 0 1 0 1 1 0 1 0 0 1 0 1 10 5 15 10 20 1 20 3 12 6 17 2 14 4 13 5 10	0 -1 -1 0 0 0 -1 0 1	

Task 3: (10 points) Implement the basic operations of a Sparse Table

Given an array of N integers, find the **GCD** value of a range [L,R] using Sparse Table.

You have to print the following information:

1. The ranges to be stored in the Sparse table
2. Print the Sparse table after construction
3. Answer the result of each query and how the answer was extracted from Sparse Table

Input:

The first line contains the value of N. The Second line provides the N integers.

The third line will contain the Q value, representing the number of queries. The following Q lines will provide the range definitions.

(for determining GCD use the function `__gcd(x,y)`)

Sample Input	Sample output
10 30 10 50 100 20 25 5 15 40 60 5 0 9 2 6 3 7 1 3 4 4	Ranges to be stored in Sparse Table: (0,0) (0,1) (0,3) (0,7) (1,1) (1,2) (1,4) (1,8) (2,2) (2,3) (2,5) (2,9) (3,3) (3,4) (3,6) (4,4) (4,5) (4,7) (5,5) (5,6) (5,8) (6,6) (6,7) (6,9) (7,7) (7,8) (8,8) (8,9) (9,9) Status of Sparse Table: 30 10 10 5 10 10 10 5 50 50 5 5 100 20 5 20 5 5 25 5 5 5 5 5 15 5 40 20 60 Query-1: GCD=5 gcd([0,7],[2,9]) Query-2: GCD=5 gcd([2,5],[3,6]) Query-3: GCD=5 gcd([3,6],[4,7]) Query-4: GCD=10 gcd([1,2],[2,3]) Query-5: GCD=20 gcd([4,4],[4,4])

Task 4: (10 points) Highly Secured Lab

The security system in a mobile lab records N security log entries, each represented as an integer. Due to the sensitivity of the logs, each entry contains bitwise information about detected anomalies. The **bitwise OR** operation between a range of logs gives the combined anomaly pattern for that time period.

To verify the integrity of the system, you are given Q verification queries. Each query checks whether the **bitwise OR** of a given range matches the i -th security log entry. If it matches, print "YES", otherwise, print "NO".

Input:

1. An integer N ($1 \leq N \leq 10^5$) – the number of security log entries.
2. N space-separated integers $A_1, A_2, \dots, A_N$ ($0 \leq A_i \leq 10^9$) – the security log entries.
3. An integer Q ($1 \leq Q \leq 10^5$) – the number of verification queries.
4. Q lines follow, each containing two integers L, R and i ($1 \leq L \leq R \leq N$)

Output:

For each query, print "YES" if the bitwise OR of the range $[L, R]$ equals X , otherwise print "NO".

Sample Input	Sample output
8 3 5 7 10 15 8 12 15 4 1 3 4 2 5 5 0 6 7 3 7 6	YES NO YES NO
8 1 0 1 0 1 0 1 0 5 0 2 2 1 3 3 2 5 5 3 6 6 4 7 7	YES NO NO YES NO

Task 5: (10 points) Budget Friendly Caraxes in Budget Friendly Westeros (Graph)

In the budget-friendly version of Westeros, King (Consort) Daemon Targaryen is set on a journey riding his dragon Caraxes. However, in this version, Caraxes doesn't fly; instead, it hops across cities in Westeros. Each time Caraxes hops, it can hop over exactly k cities in one move. Given a starting location and an ending location, your task is to determine if Daemon can reach his destination and also print the path he must follow to get there, if possible.

Input:

- The first line contains two integers V , E and Q representing the total number of cities, edge information and Queries respectively.(undirected)
- The next line will contain V strings representing names of cities in Westeros.
- The following E lines contain two city names (strings) $city1$ and $city2$ which represents an edge from $city1$ to $city2$.
- The following Q lines contain src , dst and k which represents source city, destination city and number of cities Caraxes cover in each hop respectively.

Output:

- If it is possible to reach the destination city from the source, print the path.
- Otherwise print **NO**.

Sample Input	Sample Output
12 10 KingsLanding Blackwood Winterfell Dorne Dragonstone Wall Essos Lannisport Harrenhal Riverrun RedKeep StormsEnd	
KingsLanding Blackwood KingsLanding Winterfell KingsLanding Dorne Winterfell Wall Winterfell Dragonstone Dragonstone Harrenhal Dragonstone Riverrun Dorne Essos Dorne Lannisport Essos RedKeep	
KingsLanding Harrenhal 1	KingsLanding Winterfell Dragonstone Harrenhal
KingsLanding Harrenhal 3	NO
Harrenhal RedKeep 2	Harrenhal Winterfell Dorne RedKeep
RedKeep Riverrun 3	RedKeep KingsLanding Riverrun
StormsEnd Winterfell 1	NO
Essos Dragonstone 5	NO

Note that -

1. A city can't appear twice in a path
2. There are no cycles in the graph
3. There might be some disconnected components in the graph

Task 6: (10 points) Mr. Unascendable (Graph)

The brilliant (but slightly eccentric) engineering students of the Institute of Unending Torment have created a wondrous robot named "Unascendable". This marvel of engineering can perform a dazzling array of tasks, from completing DS lab tasks in seconds to answering COA questions correctly. However, due to a design flaw involving a misplaced spring and an overabundance of enthusiasm for optimizing "downward mobility", Unascendable **cannot climb**. Not even a little bit.

Unascendable has been placed on a grid-based testing ground with varying terrain height. The grid is represented by an $m \times n$ integer matrix called `map`, where `map[r][c]` represents the height of the block at coordinate (r, c) . The top and left edges of the grid border the "Green Zone", a lush and verdant area filled with cooling pods and repair tools. The bottom and right edges border the "Red Zone", a fiery volcanic region with magma powered fast charging stations.

Unascendable can move from one block to an adjacent block (north, south, east, or west) only if the **height of the destination block is less than or equal to the height of its current block**. In other words, it can jump down or move across to blocks of the same height, but climbing is strictly off-limits.

You are given the map matrix. Determine the starting coordinates (r, c) of Unascendable from which the robot can reach both the Green Zone and the Red Zone. There can be multiple starting coordinates from which it can reach both zones. You have to find all of those coordinates and represent them in a similar map.

Input:

- The first line contains two integers m and n representing dimension of the map.
- The following lines will contain the map where each position `map[r][c]` denotes the height of the terrain in (r,c) position.

Output:

- An $m \times n$ character matrix with each cell containing either 'O' or 'X'. 'O' in the cell (i,j) means Unascendable can reach both Green and Red zones starting from (i,j) and 'X' means it can't.

Sample Input	Sample Output
3 3 2 3 3 2 1 2 3 2 2	X O O X X X O X X
5 5 1 2 2 3 5 3 2 3 4 4 2 4 5 3 1 6 7 1 4 5 5 1 1 2 4	X X X X O X X X O O X X O X X O O X X X O X X X X
4 3 1 4 1 2 3 2 3 2 3	X O O X O O O X X

4 1 4	0 X 0
1 1 1	0

Test case 1 explanation

Input

	0	1	2	
0	2	3	3	0
1	2	1	2	1
2	3	2	2	2
	0	1	2	

Output

	0	1	2	
0	X	0 (U,RR)	0(U,R)	0
1	X	X	X	1
2	0 (L,D)	X	X	2
	0	1	2	

X - Not reachable

0 - Reachable

(G,R) - G → moves to reach Green Zone, R → moves to reach Red Zone

L → Left R → Right

U → Up D → Down

Task 7: (5 points) Island Tally (Graph)

An ancient pirate map has been discovered, revealing the location of a hidden treasure buried within a vast archipelago. The map is a 2D grid where '1' marks islands and '0' marks the surrounding sea. To plan your treasure-hunting expedition, you need to determine the exact number of islands in this archipelago.

Input:

- The first line will contain two integers, m and n, separated by a space, representing the number of rows and columns of the grid, respectively.
- The following m lines will each contain a string of length n consisting of '1's and '0's. Each string represents a row in the grid.

Output:

The output will be a single integer representing the number of islands in the grid.

Note: Islands can't be diagonally connected.

Sample Input	Sample Output
5 5 11000 11000 00100 00011 10001	4
3 3 101 010 101	5
4 4 1100 1100 0011 0011	2
4 4 1010 0101 1010 0101	8
7 7 1111111 1000001 1011101 1010101 1011101 1000001 1111111	1

Task 8: Check if there exists a path between two nodes (Graph)

You are given a directed graph with N nodes and M edges. The graph is represented by pairs of integers, where each pair (a, b) denotes a directed edge from node a to node b . Your task is to determine whether there exists a path from a source node src to a destination node $dest$.

Input:

- The first line contains two integers N and M , where N is the number of nodes, and M is the number of edges.
- The next M lines each contain two integers a and b , denoting a directed edge from node a to node b .
- The last line contains two integers src and $dest$, representing the source and destination nodes.

Output:

- Print "YES" if there exists a path from src to $dest$.
- Print "NO" if there does not exist a path from src to $dest$.

Sample Input	Sample Output	Explanation
4 3 1 2 2 3 3 4 1 4	Yes	There is a path from node 1 to node 4: $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$. Therefore, the output is "YES".
5 8 1 2 1 3 2 4 2 5 4 4 5 2 4 3 3 5 5 3	Yes	5 2 4 3
5 8 1 2 1 3 2 4 2 5 4 4 5 2 4 3 3 5 2 1	No	
5 4 1 2 2 3 4 5 4 2 1 5	No	Although node 1 can reach 2, 3, and 4, there is no path from node 1 to node 5.
5 2	No	Disconnected Graph

1 2 3 4 1 5		
-------------------	--	--

Task 9: CG Supremacy (Graph)

Aziz is working hard to improve his grades in the Data Structure course, but despite his efforts, he finds himself struggling to perform as well as his peers. While it's somewhat understandable that students with higher CG than him might make fun of him, what truly bothers him is that students with lower CGs than his are also bullying him. This seems completely unjustifiable to him.

To get to the bottom of this, **Aziz** has sought your help since he's too busy completing his pending lab tasks. Your goal is to implement **Topological Sort** algorithm to figure out who can bully him, and who he can bully in return.

Input:

- The first line contains two integers V, E and Q representing the total number of students, CG relations and Queries respectively.
- The next line will contain V strings representing student names.
- The following E lines contain two names(strings) *student1* and *student2* which represents that *student2* has higher CG than *student1*.
- The following Q lines contains a pair of student names : *student1* and *student2*

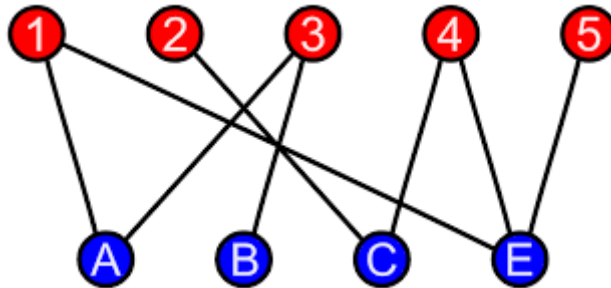
Output:

- If a valid sequence of V names exists such that each student has a higher CG than all other students before them, print the sequence (not unique). Otherwise print 'NOT POSSIBLE'.
- If such a sequence exists, print the answers to the Q queries separated by a newline where you will print **YES** if *student1* can make fun of *student2*. Otherwise print **NO**.

[illegible]

Task 10: Bipartite Graph

A bipartite graph (or bigraph) is a graph whose vertices can be divided into two disjoint and independent sets U and V , that is, every edge connects a vertex in U to one in V .



There is an **undirected** graph with n nodes, where each node is numbered between 0 and $n - 1$. The graph won't have any self or parallel edges. Given the adjacency list, determine whether the given graph is bipartite or not.

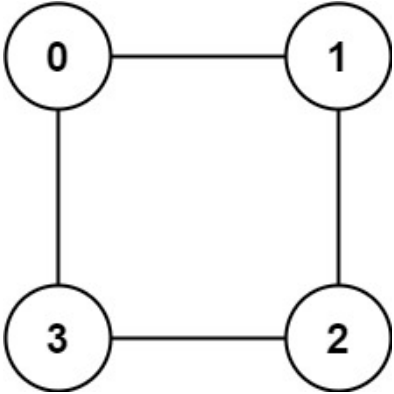
Input:

- The first line will contain a number V which denotes the number of vertices in the graph.
- The next V lines will start with a number N denoting how many vertices are connected to it followed by N vertices.

Output:

- YES if the graph is bipartite.
- NO otherwise

Sample Input	Sample Output	
4 3 1 2 3 2 0 2 3 0 1 3 2 0 2	NO	

4 2 1 3 2 0 2 2 1 3 2 0 2	YES	
4 2 1 2 2 0 2 2 0 1 1 2	NO	
9 1 1 2 0 2 2 1 3 3 2 4 6 2 3 5 2 4 8 2 3 7 2 6 8 2 5 7	YES	
6 1 1 3 0 2 5 2 1 3 2 2 4 2 3 5 2 1 4	NO	